

Project Documentation

By ARYA CHATTERJEE

Title: AI-Powered Insurance Policy Chatbot using Gemini LLM and RAG Architecture

[Video demonstration click here](#)

Overview

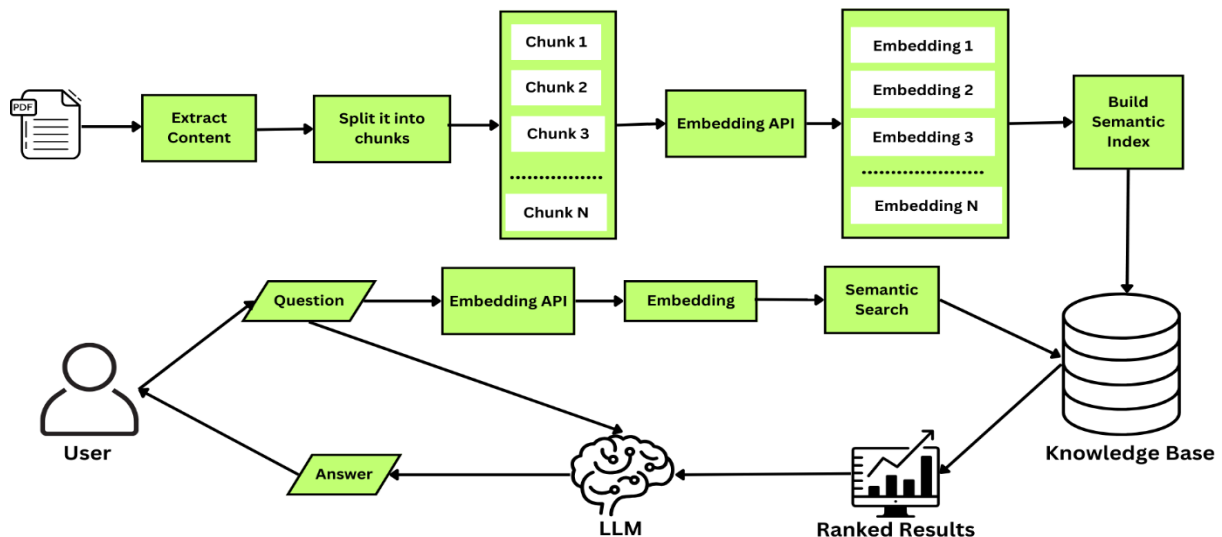
This project is an intelligent chatbot system that enables users to ask questions related to insurance policy documents. It uses a combination of semantic search and large language models (LLMs) to return accurate, context-aware answers grounded in the knowledge base extracted from PDF documents.

This system is designed with commercial deployment scalability in mind for BFSI and other document-intensive industries.

Key Features

- Automated document ingestion and processing
- Semantic understanding of queries
- Context-aware conversation handling
- Hybrid QA mechanism (retrieval + generative fallback)
- Styled output for better user experience
- Easy scalability and extendibility (e.g., web integration)

System Architecture



1. Document Ingestion and Preprocessing

- **Step:** Document Loader loads the content from PDF or text files.
 - **Process:**
 - Content is split into **overlapping chunks** using a text splitter for better semantic consistency.
 - These chunks are necessary to maintain **LLM input limitations** while preserving context.
-

2. Embedding and Indexing

- **Step:** Each chunk is passed to an **Embedding API** (Google Generative AI embeddings).
 - **Process:**
 - The API converts each chunk into **vector representations**.
 - These vectors are then stored in a **FAISS vector store**, which builds **semantic indexes** for efficient retrieval.
-

3. User Query Handling (Text/Web Interface)

- **Step:** User submits a question via the chatbot UI.
 - **Process:**
 - The user query is embedded into a vector using the same **embedding model**.
 - The query vector is then used to perform a **semantic search** in the FAISS store.
 - **Most relevant chunks** (based on vector similarity) are retrieved.
-

4. Answer Generation with LLM

- **Step:** Retrieved chunks + user query are passed to the **LLM (LLaMA 2 / Gemini)**.
- **Process:**
 - The LLM uses **Retrieval-Augmented Generation (RAG)** to combine relevant information and generate a contextually accurate answer.
 - The generated response is displayed in the **web interface**.

5. Contextualized Conversations

- **Step:** Past questions and answers are stored during the session.
- **Process:**

- When a user asks follow-up questions, the **LLM is fed with historical context** (previous 2–3 Q&As).
 - This helps in **maintaining continuity and relevance** of responses.
 - Optionally, **user feedback** can be stored to fine-tune or adapt responses dynamically (simulated fine-tuning).
-

6. Multimodal Interaction (Voice + Language Translation)

- **Voice Input:**
 - The **Whisper model** (by OpenAI) is used to convert **spoken queries to text**.
- **Response Generation:**
 - Follows the same process as text input (semantic retrieval + LLM answer).
- **Language Translation:**
 - Both input and output text can be translated using **T5 / GPT-2 models** as language translators.
- **Voice Output:**
 - Final text responses are converted back to speech using **gTTS (Google Text-to-Speech)**.

Component Level Breakdown

1. PDF Loader

- **Tool:** PyPDFLoader (from langchain.document_loaders)
 - **Responsibility:** Load text from PDF files.
 - **Details:**
 - Extracts raw text page-wise from documents.
 - Each page becomes a single Document object initially.
-

2. Text Chunking

- **Tool:** RecursiveCharacterTextSplitter
- **Responsibility:** Break long documents into smaller, manageable chunks.
- **Parameters:**

- `chunk_size = 1000`
 - `chunk_overlap = 200`
 - **Why:**
 - Ensures semantic integrity for embedding and improves retrieval granularity.
-

3. Embedding

- **Tool:** GoogleGenerativeAIEmbeddings
 - **Model:** "models/embedding-001"
 - **Responsibility:**
 - Convert each document chunk into a vector.
 - Feed these vectors into a vector store for semantic search.
-

4. Vector Store

- **Tool:** FAISS (Facebook AI Similarity Search)
 - **Responsibility:**
 - Store embeddings.
 - Perform fast similarity-based document retrieval.
-

5. LLM (Generative AI)

- **Tool:** ChatGoogleGenerativeAI
 - **Model:** "models/gemini-2.0-flash"
 - **Responsibility:**
 - Generate natural language answers.
 - Handle user conversation context.
 - Fallback to direct LLM queries if retrieval-based response is weak.
-

6. RetrievalQA Chain

- **Tool:** langchain.chains.RetrievalQA
- **Responsibility:**
 - Combine vector-based retriever with the LLM.

- Return the best possible response from semantic results.
-

7. Fallback Mechanism

- **Trigger:**
 - If answer length is too short or contains "sorry"/non-informative text.
 - **Mechanism:**
 - Use full conversation history with direct LLM invocation.
 - **Benefit:**
 - Increases coverage for ambiguous or out-of-domain queries.
-

8. Conversation Manager

- **Logic:**
 - Maintains history of last 3 interactions.
 - Ensures user context is passed in fallback mode.

9. UI/UX Layer

- **Tool:** IPython.display (Jupyter style)
 - **Feature:**
 - HTML-formatted bot response for better readability in CLI.
 - Structure includes labels, line breaks, and bullet replacements.
-

Functionality Walkthrough

1. Start

- Run `start_conversation()`
- User is greeted and prompted for input.

2. Query Processing

- Input query is passed to `ask_bot()`
- Query is vectorized and used in semantic search.
- If retrieved answer is short or vague → fallback to direct Gemini response.

3. Response Formatting

- Bot response is HTML-rendered using `format_response_html()`

- Printed in a readable conversational format.

4. Exit

- User can type exit, quit, or bye to end the session.

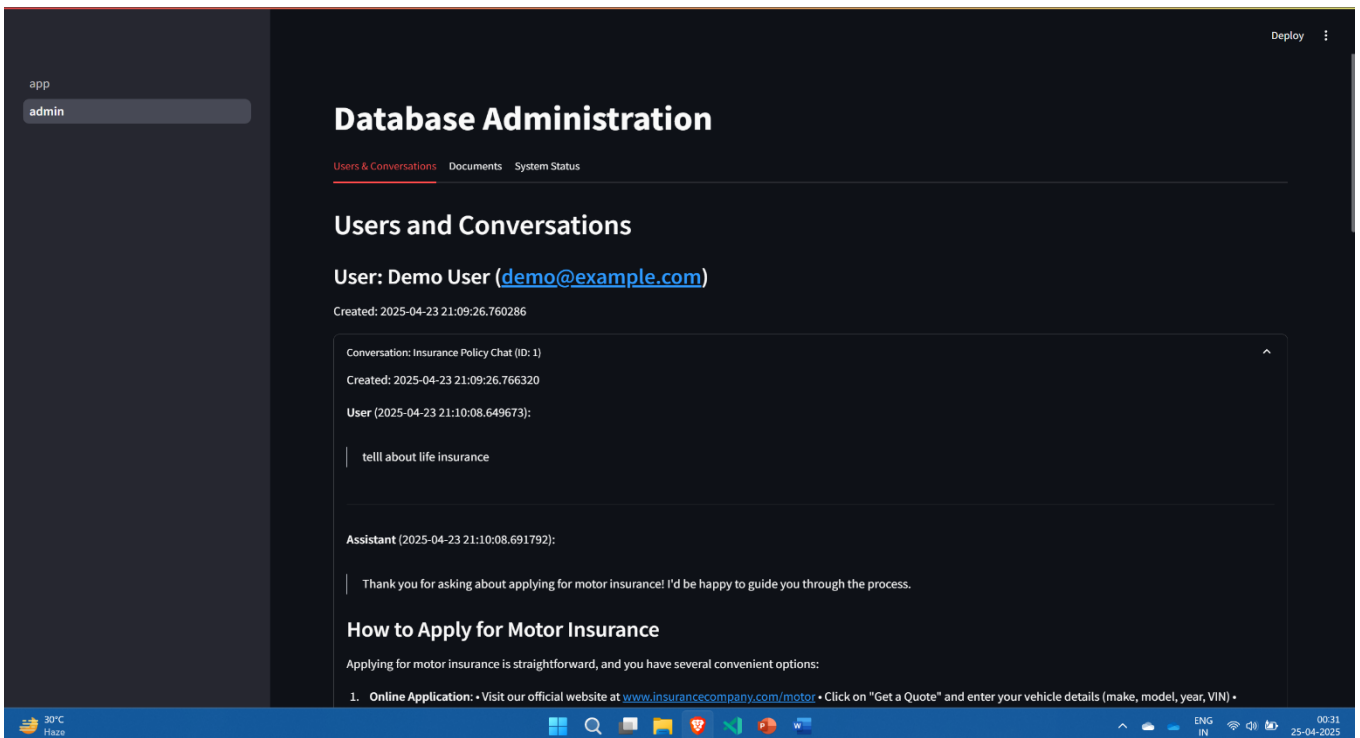
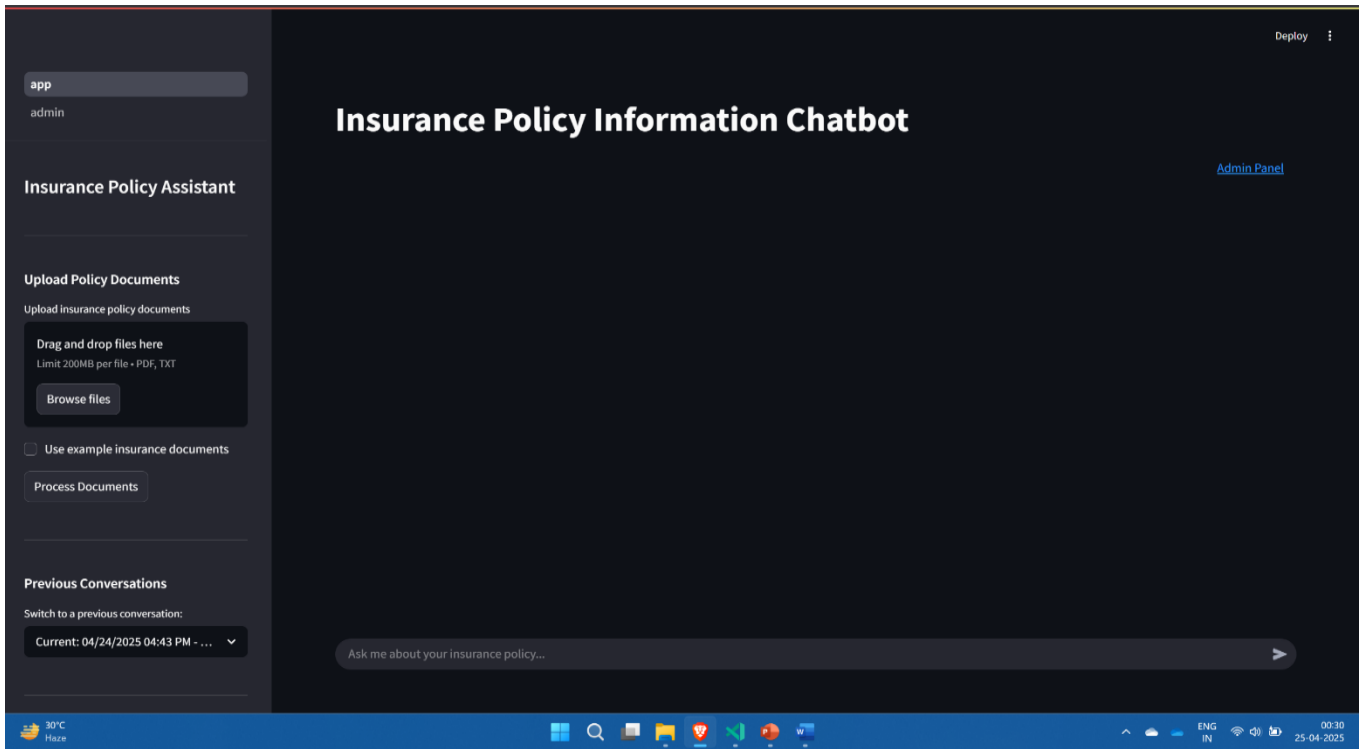
Challenges & Resolutions

- **Data Availability and Quality:** Availability of comprehensive and up-to-date information is difficult .
- **Hardware Limitations:** Using advanced models(ChatGPT, LLama), to train or deploy requires heavy Computing power.
- **API Limitations:** for deploying LLM models locally or in cloud we have to use API keys. Which can be prone to server errors and network issues.
- **Token Limit:** There is token limitation in every model which affects the length and quality of the output produced.

Challenge	Resolution / Learning
Chunk size tuning	1000 with 200 overlap was optimal.
Short/Generic responses from QA	Added fallback to LLM with context.
Context loss in conversation	Passed last 3 messages during fallback.
LLM latency	Only triggered when necessary.

Video demonstration click here : <https://youtu.be/SsOPdWvoYIs>

Chat & Admin Interface



Database & Connectivity

The screenshot shows the Astra DB web interface. The top navigation bar includes 'Docs', 'Settings', and a user profile. The left sidebar has 'Home', 'Databases', and 'Streaming' sections. The main content area is titled 'Databases' and features a 'Create Database' button. A prominent banner for 'Generate embeddings direct from Astra DB' is displayed, highlighting integrations with OpenAI, Azure OpenAI, and NVIDIA. Below this, the 'Serverless' section shows 'Database Usage Totals' for a specific database, including Read Requests (15.1k), Write Requests (46.9k), Storage Consumed (34.13 MB), and Data Transfer (70.95 MB). A table lists the database details, showing it is 'Active' and providing a 'Connect' link.

Name	Database ID	Reads	Writes	Storage	Data Transfer	Status	Actions
knowledge_.....	0457c0a2-5bb5-4c42-84a8-24da2bfeaba2	15.1k	46.9k	34.13 MB	70.95 MB	Active	Connect

The screenshot shows the 'Connection details' dialog box in the Astra DB interface. It is divided into three main sections: '2. Generate application token', '3. Connect', and 'Next Steps'. The 'Generate application token' section includes a 'Token description' field and a 'Generate token' button. The '3. Connect' section provides a code snippet for connecting to the database using the Astra Python client. The 'Next Steps' section lists actions like 'Load vector data into a collection', 'Perform a vector search', and 'View the full Python API reference'.

```
from astrapy import DataAPIClient

# Initialize the client
client = DataAPIClient("YOUR_TOKEN")
db = client.get_database_by_api_endpoint(
    "https://0457c0a2-5bb5-4c42-84a8-24da2bfeaba2-us-east-2.apps.astra.datastax.com"
)

print(f"Connected to Astra DB: {db.list_collection_names()}")
```


Document Upload & Processing

